

Bandung House Price Prediction

Bandung House Price Prediction is a regression task aimed at estimating property prices based on key housing features such as the number of bedrooms, bathrooms, land area, and building size.

```
In [1]: # Importing Libraries
import pandas as pd
```

```
In [2]: # Importing dataset
df = pd.read_csv('../data/data.csv')
df.head()
```

Out[2]:

	Rumah	Premier	Rp 2,1 Miliar	Cicilan: 7 Jutaan per bulan	Rumah 2 Lantai Bagus Sertifikat Hak Milik di Maltra Residence, Bandung	Andir, Bandung	3	2	2.1	137 m ²	170 m ²
0	Rumah	NaN	Rp 4,1 Miliar	Cicilan: 15 Jutaan per bulan	Rumah Cantik @ Andir (Sudirman)	Andir, Bandung	3	2	3	202 m ²	300 m ²
1	Rumah	NaN	Rp 3,3 Miliar	Cicilan: 12 Jutaan per bulan	Jual Cepat Rumah di Maleber Rajawali Bisa Untu...	Andir, Bandung	5	2	1	350 m ²	258 m ²
2	Rumah	NaN	Rp 580 Juta	Cicilan: 2 Jutaan per bulan	Rumah minimalis 3 lantai di andir Bandung coco...	Andir, Bandung	2	2	1	30 m ²	80 m ²
3	Rumah	NaN	Rp 1,3 Miliar	Cicilan: 4 Jutaan per bulan	Jual Rumah Murah Bisa Dijadikan Kos"an Di Jala...	Andir, Bandung	11	3	0	176 m ²	176 m ²
4	Rumah	NaN	Rp 3,6 Miliar	Cicilan: 13 Jutaan per bulan	Rumah 2,5 Lantai Strategis SHM di Komplek Kert...	Andir, Bandung	5	3	1	184 m ²	234 m ²

Data Preprocessing Part 1

```
In [3]: # Checking the shape of dataset
df.shape
```

```
Out[3]: (7610, 11)
```

```
In [4]: # Removed unused column
un_col = ['Rumah', 'Premier', 'Cicilan: 7 Jutaan per bulan']
df.drop(un_col, axis=1, inplace=True)
df.columns
```

```
Out[4]: Index(['Rp 2,1 Miliar',
              'Rumah 2 Lantai Bagus Sertifikat Hak Milik di Maltra Residence, Bandung',
              'Andir, Bandung', '3', '2', '2.1', '137 m²', '170 m²'],
             dtype='object')
```

```
In [5]: # Rename column
df.rename(columns={'Rp 2,1 Miliar': 'price',
                  'Rumah 2 Lantai Bagus Sertifikat Hak Milik di Maltra Residence,
                  Andir, Bandung' : 'location',
                  '3' : 'bedroom',
                  '2' : 'bathroom',
                  '2.1' : 'carport',
                  '137 m²' : 'land_size',
                  '170 m²' : 'building_size',
                  },
          inplace=True)
df.columns
```

```
Out[5]: Index(['price', 'name', 'location', 'bedroom', 'bathroom', 'carport',
              'land_size', 'building_size'],
             dtype='object')
```

```
In [6]: # Check duplicate
df.duplicated().sum()
```

```
Out[6]: np.int64(616)
```

```
In [7]: # Remove duplicate
df.drop_duplicates(inplace=True)
```

Property Type Labeling

After exploring the dataset, some listings were identified as invalid property types based on their name (e.g., not actual houses). So i use a simple text classification approach to categorize them.

a rule-based text classification approach was applied to automatically categorize listings:

1. Listings containing words like **“rumah”** or **“rmh”** were labeled as **house** .
2. Any record initially labeled as **house** but containing commercial keywords such as **“kost”**, **“ruko”**, **“villa”**, or **“hotel”** was reclassified as **non_house** .

This simple rule-based approach was chosen intentionally because the main goal of this project is to **build a price prediction model**, not to develop a complex text classification system.

Therefore, an efficient and interpretable filtering method was sufficient to separate residential (**house**) listings from commercial (**non_house**) ones before modeling.

While this approach effectively distinguishes most property types, it has one limitation: **listings that do not explicitly mention “rumah” or any known property type** (e.g., titles containing only marketing terms or locations) **cannot be accurately detected** and may remain misclassified.

```
In [8]: import numpy as np

# Make new column 'type' as property type and fill the values base on column 'name'
df['type'] = np.where(df['name'].str.contains('rumah|rmh' , case=False, na=False),
                     'house',
                     'non_house')
```

```
In [9]: # a word that usually signifies not a house
keywords_non_house = ['ruko', 'kost', 'kos ', 'kos' , 'hotel', 'villa', 'gedung', '

# override the incorrect results
mask_house = df['type'] == 'house'
mask_invalid = df['name'].str.contains('|'.join(keywords_non_house), case=False, na

# change the type into 'non_house' if the conditions are True
df.loc[mask_house & mask_invalid, 'type'] = 'non_house'

df['type'].value_counts()
```

```
Out[9]: type
house      6250
non_house   744
Name: count, dtype: int64
```

```
In [10]: df[df['type'] == 'non_house'].head(5)
```

Out[10]:

	price	name	location	bedroom	bathroom	carport	land_size	building_size	
3	Rp 1,3 Miliar	Jual Rumah Murah Bisa Dijadikan Kos"an Di Jala...	Andir, Bandung	11	3	0	176 m ²	176 m ²	non_
46	Rp 2,57 Miliar	Strategis Ruko 2 Lantai Area Rajawali Garuda A...	Andir, Bandung	4	2	2	134 m ²	220 m ²	non_
48	Rp 3 Miliar	Dijual Rumaah di Lugina Andir	Andir, Bandung	2	1	1	240 m ²	200 m ²	non_
53	Rp 400 Juta	Rumah ex kos2 an jl Maleber Barat	Andir, Bandung	5	3	0	56 m ²	95 m ²	non_
95	Rp 2,25 Miliar	Rumah Kost 3 Lantai Di Ciroyom Andir Bandung	Andir, Bandung	11	5	0	163 m ²	300 m ²	non_

```
In [11]: # Remove non-house listings row
df = df[df['type'] != 'non_house']
```

String & Numeric Format Cleaning

```
In [12]: # Make new Price column
df['price_clean'] = (
    df['price']
    .str.replace('[^0-9,\.]', '', regex=True)
    .str.replace(',', '.', regex=False)
)

# change Price data type to numeric
df['price_clean'] = pd.to_numeric(df['price_clean'], errors='coerce')

# If Condition for 'Miliar' and 'Juta'
```

```
df.loc[df['price'].str.contains('miliar', case=False, na=False), 'price_clean'] = d
df.loc[df['price'].str.contains('juta', case=False, na=False), 'price_clean'] = df[
```

```
In [13]: # Make new land_size column
df['land_size_clean'] = (
    df['land_size']
    .astype(str)
    .str.replace('[^0-9,\.]', '', regex=True)
    .str.replace(',', '.', regex=False)
)

# Change land_size data type to numeric
df['land_size_clean'] = pd.to_numeric(df['land_size_clean'], errors='coerce')
```

```
In [14]: # Make new building_size column
df['building_size_clean'] = (
    df['building_size']
    .astype(str)
    .str.replace('[^0-9,\.]', '', regex=True)
    .str.replace(',', '.', regex=False)
)

# change building_size data type to numeric
df['building_size_clean'] = pd.to_numeric(df['building_size_clean'], errors='coerce')
```

```
In [15]: # Removed unused columns
un_col = ['name', 'location', 'price', 'land_size', 'building_size', 'type']
df.drop(un_col, axis=1, inplace=True)
df.columns
```

```
Out[15]: Index(['bedroom', 'bathroom', 'carport', 'price_clean', 'land_size_clean',
               'building_size_clean'],
              dtype='object')
```

```
In [16]: # Check data type
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 6250 entries, 0 to 7609
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   bedroom               6250 non-null   int64
1   bathroom              6250 non-null   int64
2   carport               6250 non-null   int64
3   price_clean           6250 non-null   float64
4   land_size_clean       6249 non-null   float64
5   building_size_clean   6250 non-null   int64
dtypes: float64(2), int64(4)
memory usage: 470.8 KB
```

```
In [17]: # check missing values
df.isnull().sum()
```

```
Out[17]: bedroom          0  
         bathroom        0  
         carport         0  
         price_clean      0  
         land_size_clean  1  
         building_size_clean 0  
         dtype: int64
```

```
In [18]: # delete missing values  
df.dropna(inplace=True)
```

```
In [19]: # checking number of unique value of each column  
df.nunique()
```

```
Out[19]: bedroom          24  
         bathroom        17  
         carport         19  
         price_clean      639  
         land_size_clean  699  
         building_size_clean 432  
         dtype: int64
```

```
In [20]: #checking the unique values in each column  
cols = df.columns  
for i in cols:  
    print(i, df[i].unique(), '\n')
```

bedroom [3 5 2 4 6 7 0 9 8 1 12 13 11 10 19 14 18 16 15 27 23 24 29 40]

bathroom [2 3 1 5 0 4 7 11 6 10 8 9 13 17 12 14 40]

carport [3 1 2 6 7 0 4 5 8 10 30 9 15 20 13 16 12 11 14]

price_clean [4.10e+09 3.30e+09 5.80e+08 3.60e+09 1.17e+09 2.90e+09 4.50e+09 2.10e+09
2.20e+09 1.40e+09 7.00e+09 7.50e+09 5.80e+09 3.50e+09 1.50e+09 8.00e+09
6.50e+08 3.90e+09 1.20e+09 1.10e+09 2.75e+09 1.05e+10 2.30e+09 8.75e+08
5.50e+09 1.75e+09 1.80e+09 4.25e+09 6.00e+08 2.65e+08 2.35e+09 5.00e+08
5.50e+08 1.25e+09 1.60e+09 1.19e+10 6.50e+09 2.00e+09 9.00e+09 8.90e+09
1.35e+09 3.50e+10 4.00e+09 1.58e+09 1.10e+10 3.25e+08 5.30e+09 2.50e+09
9.99e+08 1.99e+09 1.00e+09 3.70e+09 5.00e+09 3.95e+09 4.75e+09 1.30e+09
1.55e+09 4.95e+09 1.50e+10 6.25e+09 8.50e+09 1.90e+10 2.60e+08 3.00e+09
1.70e+10 1.24e+09 7.90e+09 3.25e+09 1.65e+09 4.20e+09 8.90e+08 5.60e+09
9.50e+09 2.30e+10 1.15e+09 8.00e+08 1.98e+09 1.70e+09 2.10e+10 8.25e+09
1.90e+09 5.40e+09 3.10e+09 1.85e+10 6.57e+08 3.75e+09 3.00e+08 5.75e+09
2.25e+09 1.37e+10 8.50e+08 3.80e+09 1.45e+09 5.12e+08 7.00e+08 1.62e+09
6.48e+08 3.75e+08 3.39e+09 6.00e+09 1.37e+09 9.50e+08 4.00e+08 3.20e+09
4.50e+08 1.33e+09 9.68e+08 2.00e+10 4.15e+09 6.85e+09 1.95e+09 8.65e+08
1.20e+10 3.40e+09 9.85e+08 9.58e+08 2.60e+09 2.85e+09 1.50e+08 3.50e+08
1.85e+09 3.30e+10 6.60e+09 1.57e+09 6.90e+08 7.50e+08 2.40e+09 5.60e+08
3.80e+10 6.25e+08 3.85e+09 1.00e+10 7.50e+07 7.25e+08 2.15e+09 3.20e+08
3.05e+09 4.30e+09 6.85e+08 2.70e+09 7.75e+08 9.00e+08 8.80e+09 7.70e+09
1.97e+09 8.15e+08 7.75e+09 4.70e+09 1.30e+10 1.70e+08 5.25e+08 1.34e+09
2.59e+09 1.39e+09 3.15e+09 1.52e+09 7.85e+08 7.99e+08 4.40e+09 2.95e+09
4.80e+08 8.49e+08 1.44e+09 9.35e+08 1.22e+09 9.75e+08 7.80e+08 8.30e+08
1.27e+09 6.35e+09 4.70e+08 7.60e+08 2.14e+09 4.90e+08 1.19e+09 1.29e+09
8.45e+08 6.44e+08 2.58e+09 6.82e+08 2.80e+09 6.75e+08 6.10e+08 1.07e+09
1.79e+09 1.11e+09 9.90e+08 8.10e+08 1.49e+09 6.70e+08 7.95e+08 7.70e+08
1.12e+09 7.94e+08 7.93e+08 1.64e+09 9.15e+08 5.85e+08 9.45e+08 1.89e+09
7.90e+08 3.65e+09 1.48e+09 9.57e+08 5.59e+08 6.30e+08 1.69e+09 7.91e+08
5.40e+08 2.10e+08 5.95e+08 2.35e+08 3.24e+08 6.80e+08 5.97e+08 6.45e+08
6.99e+08 9.25e+08 2.32e+09 6.95e+08 4.85e+08 8.99e+08 8.25e+08 4.53e+08
4.75e+08 4.35e+08 4.60e+09 2.99e+09 1.09e+10 8.60e+08 2.45e+09 6.30e+09
5.30e+08 1.38e+09 2.82e+08 3.92e+08 2.55e+09 9.30e+09 6.20e+09 5.20e+09
1.45e+10 1.65e+08 2.05e+09 2.05e+10 6.05e+08 2.22e+09 9.20e+08 1.12e+10
2.65e+09 5.10e+09 9.80e+08 3.03e+09 8.80e+08 4.65e+09 1.80e+08 1.92e+09
5.70e+09 5.25e+09 9.20e+09 2.20e+10 7.35e+11 9.75e+09 7.99e+09 1.32e+09
1.02e+09 1.87e+09 2.50e+10 1.05e+09 1.75e+08 5.90e+08 7.78e+08 1.35e+08
9.95e+08 1.72e+09 5.45e+08 1.47e+09 3.07e+08 7.50e+10 2.63e+08 3.49e+09
1.25e+10 2.65e+10 1.39e+10 6.90e+09 8.74e+09 2.80e+10 4.99e+09 2.70e+10
2.20e+07 4.90e+09 1.80e+10 5.20e+10 4.00e+10 4.60e+10 1.60e+10 2.35e+10
3.60e+10 5.50e+10 3.00e+10 1.65e+10 9.90e+09 2.15e+10 6.00e+10 4.50e+10
1.40e+10 2.71e+10 5.40e+10 1.55e+10 2.60e+10 6.80e+09 3.20e+10 7.20e+09
3.40e+10 4.20e+10 4.70e+10 1.75e+10 8.70e+09 1.23e+10 3.70e+10 7.25e+09
8.78e+09 8.85e+09 3.38e+10 8.20e+08 3.35e+10 5.70e+10 3.55e+10 1.98e+10
1.35e+10 7.10e+09 2.07e+10 3.97e+09 1.86e+09 3.37e+09 4.80e+09 1.59e+09
3.59e+09 5.95e+09 6.75e+09 4.85e+09 5.85e+09 7.59e+09 7.60e+09 4.35e+09
6.70e+09 1.25e+08 4.45e+09 1.60e+08 2.89e+09 1.93e+09 4.72e+09 1.67e+10
4.25e+08 3.45e+09 1.17e+10 3.99e+09 7.77e+09 3.35e+09 4.55e+09 5.35e+09
8.70e+08 5.15e+09 4.69e+09 3.42e+09 2.97e+09 7.30e+09 9.70e+09 3.85e+08
5.87e+08 1.18e+09 8.40e+08 1.01e+09 1.42e+09 9.84e+08 1.04e+09 2.19e+08
2.79e+09 5.57e+09 3.69e+09 1.94e+09 2.78e+09 3.36e+09 2.66e+09 3.79e+09
2.29e+09 4.30e+08 4.66e+08 8.95e+08 1.28e+09 3.80e+08 1.51e+09 5.52e+08
7.51e+08 2.25e+08 4.40e+08 5.32e+08 2.85e+08 1.85e+08 3.65e+08 3.10e+08

```

4.39e+08 3.30e+08 1.84e+08 3.40e+08 2.74e+08 5.88e+08 4.84e+08 2.99e+08
2.05e+08 2.00e+08 3.99e+08 4.60e+08 2.45e+08 1.90e+08 1.45e+08 4.95e+08
5.20e+08 6.15e+08 5.75e+08 5.39e+08 6.72e+08 7.57e+08 4.20e+08 4.65e+08
8.98e+08 6.04e+08 5.99e+08 3.49e+08 3.35e+08 5.76e+08 2.75e+08 4.99e+08
7.20e+11 3.90e+08 9.70e+08 1.66e+08 3.69e+08 2.97e+08 8.35e+08 2.95e+08
2.70e+08 7.40e+08 1.99e+08 2.80e+08 6.62e+08 7.38e+08 7.35e+08 4.32e+08
4.45e+08 1.99e+10 3.83e+10 9.80e+09 5.93e+10 6.20e+10 2.75e+11 1.95e+10
7.80e+09 2.25e+10 1.42e+10 3.90e+10 8.49e+09 1.08e+10 2.37e+09 4.59e+09
1.52e+10 2.98e+09 1.05e+11 7.19e+09 1.29e+10 4.90e+10 2.82e+09 1.72e+10
6.40e+09 7.95e+09 6.49e+08 3.18e+10 6.76e+08 6.20e+08 6.35e+08 2.48e+10
5.99e+09 8.30e+09 2.43e+09 6.05e+10 2.39e+10 1.49e+10 5.97e+09 3.10e+10
1.66e+09 2.40e+10 1.02e+10 1.48e+10 4.95e+10 7.95e+10 2.25e+11 1.26e+11
2.45e+10 2.77e+10 3.27e+09 2.29e+10 8.10e+09 5.00e+10 4.00e+07 8.15e+09
2.75e+10 2.71e+09 4.33e+08 5.70e+08 9.52e+08 7.55e+08 5.45e+09 1.78e+09
1.67e+09 2.39e+09 2.15e+08 4.77e+09 4.56e+08 3.51e+08 2.35e+07 9.55e+08
2.02e+08 1.62e+08 9.01e+08 9.85e+09 6.15e+09 1.76e+09 1.14e+09 1.27e+10
5.62e+09 1.71e+09 2.63e+09 1.28e+10 5.65e+08 8.55e+09 2.11e+10 8.75e+09
6.69e+09 9.87e+09 1.87e+10 3.41e+08 5.90e+09 1.15e+10 5.48e+09 6.48e+09
2.16e+08 3.24e+09 1.54e+09 5.28e+08 5.34e+08 7.20e+08 1.63e+09 4.54e+08
5.74e+08 5.78e+08 5.82e+08 4.17e+08 4.73e+08 5.49e+09 6.81e+08 6.55e+08
5.14e+08 4.79e+08 3.86e+08 6.17e+08 5.57e+08 6.38e+08 8.55e+08 8.34e+08
6.59e+08 7.65e+08 5.56e+08 5.96e+08 1.08e+09 7.33e+08 8.57e+08 2.13e+09
6.11e+08 9.49e+08 4.50e+07 9.89e+08 8.89e+08 9.09e+08 2.40e+08 1.09e+09
5.55e+08 8.46e+08 4.89e+08 7.15e+08 2.19e+09 6.45e+09 6.18e+08 2.85e+10
3.09e+09 3.02e+09 7.10e+08 8.95e+09 2.21e+09 9.50e+07 5.77e+08 6.47e+08
2.50e+08 8.20e+09 1.18e+10 3.94e+09 2.52e+09 2.68e+10 8.40e+09 2.47e+09
7.40e+09 5.13e+09 1.38e+10 1.22e+10 1.79e+10 2.90e+10 8.85e+08 2.01e+10
3.70e+08 7.49e+08 6.39e+08 2.30e+08 1.30e+08 4.88e+08 6.93e+08 4.55e+08
4.72e+08 4.91e+08 2.90e+08 5.04e+08 5.64e+08 7.36e+08 2.52e+08 9.59e+08
4.04e+09 2.42e+09 4.38e+10 1.15e+08 2.08e+09 9.30e+08 4.89e+09]

```

```

land_size_clean [ 202.  350.   30.  184.   65.  260.  404.  137.  172.  188.  268.
 400.

```

```

222.   74.  399.  187.  180.  270.   38.  650.  170.   97.  245.  240.
660.  126.  100.  394.  233.  159.   63.  143.  269.  158.   85.   22.
198.  120.   31.   72.  200.   70.   89.  598.  364.  718.  161.  447.
317.  125.   49.  178.  510.  168.   39.  104.  153.  392.  228.  235.
140.  160.  405.  127.  129.  550.  475.  195.  255.   71. 1210.  316.
625.  500.  760.  580.  600.  439.  253.   32.  196.  770.  216.  204.
151.  655.  665.   78.   87.  107.  391. 1012.   41.  102.  891.  142.
 35.  346.  110.  348. 1016.  275.  205.  780.  470.   40.  219.  274.
 98.  138.  183.  210.  156.   96.   75.  150.  189.  300.  745.  103.
370.  389.  122.  243.   90.  139.  136.  111.   61.  612.  280.  250.
 84.  177.  414.  306.  145.  135.  483.  147.  251.  239.  312.   76.
305.  134.  311.  328.   99.   50.  118.  101.  234.   91.  106.  157.
 60.  181.  221.  225.   82.   94.  915.  131.  121.  421.  528.  214.
254.  313.  355.  130.  128.   55.  212.  504.  730.   44.  310.  297.
589.  112.  144.   66.   79.   59.  108.   83.  133.  109.  169.  116.
298.  538.  249.  154.  670.  242.  449.  800.   86.  105.  320.  267.
162.   54.   26.  380.  197.  474.  271.  330.  534.   46.  835.  256.
115.  174.  215.  308.  230.   93.  173.  237.  360.  476.   80.   95.
171.  369.   68.  186.   64.  397.  113. 1200.   52.  176.  114.  410.
327.  117.  146.   92.  220.  359.  152.  185.  353.  424.  294.   88.
287.  480.  163.  283.  119.  132.  155.  207.  477.   69.  623.  673.
124.  149.  385.  290.  356.  371.  224.  208.  560.  148.  513.  244.
278.  334.  319. 1277.  375. 1100.  547.  446.  258.  540.  450.  980.

```


325. 318. 343. 236. 868. 442. 981. 81. 420. 296. 333. 527.
 226. 473. 337. 458. 286. 408. 190. 192. 199. 390. 352. 536.
 303. 289. 459. 182. 1199. 53. 1094. 664. 512. 179. 340. 36.
 1237. 77. 653. 440. 373. 252. 4550. 493. 67. 531. 273. 485.
 871. 409. 227. 437. 880. 299. 284. 463. 292. 930. 659. 1130.
 638. 662. 843. 911. 265. 1186. 645. 649. 739. 746. 1050. 864.
 607. 543. 681. 877. 1791. 341. 345. 728. 1332. 451. 1416. 1464.
 1066. 407. 515. 347. 175. 654. 694. 755. 465. 488. 753. 445.
 610. 725. 323. 876. 1515. 520. 892. 367. 712. 508. 734. 794.
 218. 850. 881. 672. 314. 818. 42. 209. 468. 354. 1460. 1500.
 285. 123. 425. 570. 279. 262. 507. 164. 288. 396. 715. 211.
 587. 165. 238. 141. 411. 315. 1010. 282. 332. 487. 276. 916.
 326. 206. 166. 702. 433. 1205. 617. 58. 784. 377. 700. 365.
 524. 563. 324. 1110. 223. 45. 542. 805. 295. 62. 535. 73.
 191. 167. 786. 1074. 344. 48. 56. 533. 57. 281. 415. 1324.
 321. 1561. 574. 418. 521. 277. 910. 402. 885. 443. 241. 616.
 896. 453. 248. 808. 432. 435. 1405. 490. 309. 671. 675. 1400.
 884. 603. 51. 427. 462. 395. 913. 596. 954. 1268. 382. 213.
 1960. 842. 1000. 750. 430. 630. 932. 2503. 688. 505. 388. 1600.
 357. 585. 584. 1043. 801. 349. 900. 620. 631. 546. 1345. 264.
 6072. 1290. 302. 2362. 387. 339. 826. 790. 9597. 1153. 1385. 696.
 1575. 342. 1768. 194. 557. 501. 787. 577. 525. 898. 232. 1738.
 460. 554. 1295. 822. 772. 743. 246. 995. 568. 2122. 5468. 2822.
 444. 830. 748. 466. 874. 301. 1030. 1273. 661. 988. 865. 720.
 602. 1276. 678. 1024. 735. 495. 201. 2131. 680. 1014. 555. 293.
 983. 523. 1260. 467. 376. 1424. 478. 775. 261. 848. 687. 203.
 257. 779. 472. 514. 1007. 590. 401. 329. 509. 322. 455. 358.
 419. 886. 564. 860. 33. 3000. 229. 266. 413. 571. 193. 1510.
 719. 499. 441. 406. 217. 611. 384. 742. 878. 740. 336. 870.
 368. 825. 372. 689. 691. 428. 998. 1061. 426. 494. 621. 247.
 713. 908. 1015. 537. 291. 947. 398. 989. 2904. 498. 595. 1580.
 1547. 416. 575. 366. 765. 3040. 511. 526. 572. 626. 605. 1750.
 768. 1044. 1048. 990. 738. 361. 905. 964. 565. 553. 838. 862.
 544. 732. 1035. 1280. 506. 1720. 2000. 705. 1282. 454. 456. 393.
 1348. 5206. 647.]

building_size_clean [300 258 80 234 45 260 250 170
 220

160 700 400 196 78 309 370 90 450
 114 200 350 190 98 60 420 230 130
 63 288 439 150 46 175 110 77 65
 116 145 550 500 180 1000 100 115 72
 70 165 52 125 525 167 226 360 120
 320 750 457 574 280 800 557 273 128
 35 964 94 970 126 1500 140 205 900
 39 40 18 87 138 96 55 1600 168
 240 304 330 105 68 503 210 235 162
 600 132 225 650 264 71 192 137 153
 36 245 405 124 324 99 265 238 616
 88 297 112 339 109 270 266 85 195
 75 204 520 284 630 325 375 122 156
 430 365 50 754 173 56 51 202 95
 237 155 84 281 83 144 275 197 152
 148 212 43 394 121 111 198 314 239
 134 2500 41 159 135 108 92 185 154
 177 133 54 38 164 371 104 433 48

| | | | | | | | | |
|------|---------|------|------|-----|------|------|------|------|
| 136 | 2000 | 82 | 171 | 490 | 209 | 118 | 254 | 950 |
| 149 | 560 | 340 | 127 | 432 | 30 | 206 | 925 | 290 |
| 218 | 756 | 414 | 380 | 169 | 249 | 678 | 211 | 42 |
| 625 | 102 | 158 | 216 | 76 | 303 | 142 | 117 | 1199 |
| 58 | 462 | 555 | 552 | 207 | 79 | 81 | 181 | 113 |
| 521 | 279 | 313 | 437 | 403 | 770 | 587 | 463 | 6265 |
| 390 | 435 | 182 | 269 | 328 | 251 | 62 | 305 | 603 |
| 186 | 236 | 440 | 875 | 389 | 580 | 1100 | 253 | 1350 |
| 561 | 610 | 232 | 302 | 485 | 486 | 1 | 1023 | 825 |
| 295 | 214 | 268 | 208 | 425 | 296 | 890 | 480 | 961 |
| 203 | 488 | 97 | 267 | 535 | 246 | 146 | 101 | 812 |
| 252 | 276 | 67 | 222 | 410 | 344 | 278 | 262 | 248 |
| 66 | 666 | 166 | 508 | 147 | 376 | 176 | 106 | 1440 |
| 129 | 482 | 347 | 119 | 348 | 61 | 21 | 22 | 31 |
| 25 | 261 | 69 | 37 | 760 | 187 | 86 | 1400 | 353 |
| 1200 | 385 | 364 | 368 | 285 | 540 | 635 | 219 | 174 |
| 73 | 256 | 215 | 374 | 157 | 91 | 349 | 299 | 975 |
| 274 | 343 | 361 | 224 | 327 | 143 | 475 | 244 | 378 |
| 1300 | 131 | 178 | 346 | 822 | 487 | 243 | 293 | 901 |
| 510 | 660 | 257 | 352 | 570 | 221 | 460 | 223 | 172 |
| 388 | 89 | 1126 | 103 | 49 | 213 | 670 | 413 | 247 |
| 417 | 336 | 461 | 465 | 456 | 283 | 255 | 1470 | 548 |
| 595 | 64 | 998 | 478 | 184 | 33 | 2200 | 384 | 161 |
| 188 | 201 | 139 | 107 | 183 | 372 | 24 | 416 | 334 |
| 53 | 228 | 123 | 422 | 948 | 199 | 1033 | 407 | 93 |
| 936 | 765 | 1920 | 310 | 193 | 386 | 472 | 401 | 662 |
| 585 | 263 | 241 | 679 | 775 | 382 | 424 | 434 | 354 |
| 404 | 357 | 442 | 1333 | 725 | 459 | 458 | 399 | 26 |
| 326 | 3006000 | 287 | 2240 | 615 | 3625 | 312 | 227 | 624] |

In [21]: `df.describe()`

Out[21]:

| | bedroom | bathroom | carport | price_clean | land_size_clean | building_size_c |
|--------------|-------------|-------------|-------------|--------------|-----------------|-----------------|
| count | 6249.000000 | 6249.000000 | 6249.000000 | 6.249000e+03 | 6249.000000 | 6.249000 |
| mean | 3.761402 | 2.645383 | 1.324372 | 4.753621e+09 | 254.152504 | 7.118963 |
| std | 2.070665 | 1.495949 | 1.623466 | 1.563327e+10 | 319.882564 | 3.802398 |
| min | 0.000000 | 0.000000 | 0.000000 | 2.200000e+07 | 22.000000 | 1.000000 |
| 25% | 3.000000 | 2.000000 | 0.000000 | 1.100000e+09 | 100.000000 | 1.000000 |
| 50% | 3.000000 | 2.000000 | 1.000000 | 2.150000e+09 | 162.000000 | 1.700000 |
| 75% | 4.000000 | 3.000000 | 2.000000 | 4.500000e+09 | 300.000000 | 3.000000 |
| max | 40.000000 | 40.000000 | 30.000000 | 7.350000e+11 | 9597.000000 | 3.006000 |

In [22]: `(df['bedroom'] == 0).sum(), (df['bathroom'] == 0).sum()`Out[22]: `(np.int64(150), np.int64(127))`

```
In [23]: df = df[(df['bedroom'] != 0) & (df['bathroom'] != 0)]
```

After conducting an initial exploration, it was found that the bedroom and bathroom columns had a minimum value of 0, which is logically impossible for a house. Sample checks showed that the data was not for shop houses or warehouses, but rather for high-priced residential properties, so the value 0 in both columns was considered an input error or incorrectly filled missing value. Therefore, rows with a bedroom or bathroom value of 0 were deleted so that the analysis and model would not be biased by invalid data.

```
In [24]: df.describe()
```

```
Out[24]:
```

| | bedroom | bathroom | carport | price_clean | land_size_clean | building_size_c |
|--------------|-------------|-------------|-------------|--------------|-----------------|-----------------|
| count | 6081.000000 | 6081.000000 | 6081.000000 | 6.081000e+03 | 6081.000000 | 6.081000 |
| mean | 3.854136 | 2.699720 | 1.337280 | 4.680177e+09 | 252.373787 | 7.239650 |
| std | 2.011056 | 1.461777 | 1.613229 | 1.539222e+10 | 312.652255 | 3.854562 |
| min | 1.000000 | 1.000000 | 0.000000 | 2.200000e+07 | 22.000000 | 1.000000 |
| 25% | 3.000000 | 2.000000 | 0.000000 | 1.100000e+09 | 100.000000 | 1.000000 |
| 50% | 3.000000 | 2.000000 | 1.000000 | 2.150000e+09 | 162.000000 | 1.700000 |
| 75% | 4.000000 | 3.000000 | 2.000000 | 4.500000e+09 | 300.000000 | 3.000000 |
| max | 40.000000 | 40.000000 | 30.000000 | 7.350000e+11 | 9597.000000 | 3.006000 |

EDA

In the exploratory data analysis, I will be visualizing the data to get a better understanding of the data and to see if there are any trends or pattern in the data. First I will begin with looking at the distribution of the data and then I will look at the relationship between the independent variables and the target variable.

Property Feature

```
In [25]: import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# Set style
sns.set(style="whitegrid")

# Create figure layout (3 rows x 3 columns)
fig, ax = plt.subplots(2, 3, figsize=(16, 10))
```

```

# Property Features
sns.countplot(x='bedroom', data=df, ax=ax[0,0], color='skyblue')
ax[0,0].set_title('Bedroom Distribution')
ax[0,0].set_xlabel('Number of Bedrooms')
ax[0,0].set_ylabel('Count')

sns.countplot(x='bathroom', data=df, ax=ax[0,1], color='skyblue')
ax[0,1].set_title('Bathroom Distribution')
ax[0,1].set_xlabel('Number of Bathrooms')
ax[0,1].set_ylabel('Count')

sns.countplot(x='carport', data=df, ax=ax[0,2], color='skyblue')
ax[0,2].set_title('Carport Distribution')
ax[0,2].set_xlabel('Number of Carports')
ax[0,2].set_ylabel('Count')

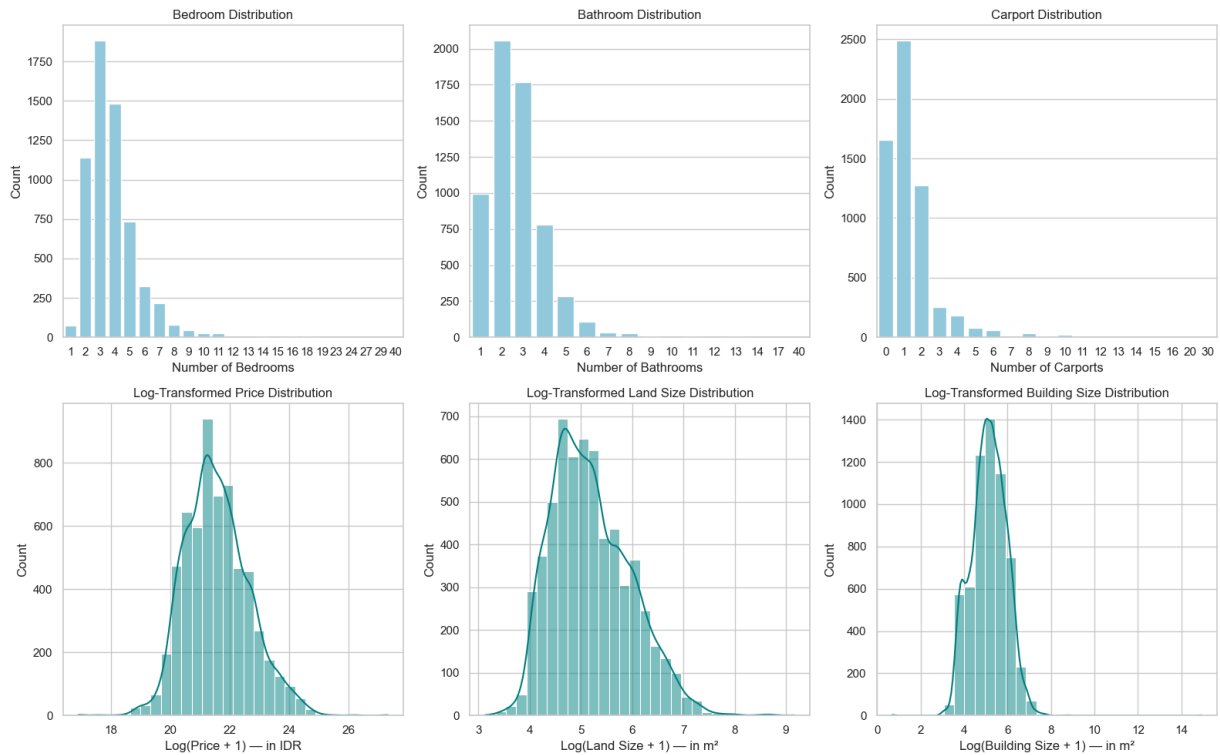
# ===== Financial Features (Log-transformed) =====
sns.histplot(np.log1p(df['price_clean']), bins=30, kde=True, ax=ax[1,0], color='teal')
ax[1,0].set_title('Log-Transformed Price Distribution')
ax[1,0].set_xlabel('Log(Price + 1) - in IDR')
ax[1,0].set_ylabel('Count')

# ===== Land & Building Size (Log-transformed) =====
sns.histplot(np.log1p(df['land_size_clean']), bins=30, kde=True, ax=ax[1,1], color='teal')
ax[1,1].set_title('Log-Transformed Land Size Distribution')
ax[1,1].set_xlabel('Log(Land Size + 1) - in m²')
ax[1,1].set_ylabel('Count')

sns.histplot(np.log1p(df['building_size_clean']), bins=30, kde=True, ax=ax[1,2], color='teal')
ax[1,2].set_title('Log-Transformed Building Size Distribution')
ax[1,2].set_xlabel('Log(Building Size + 1) - in m²')
ax[1,2].set_ylabel('Count')

plt.tight_layout()
plt.show()

```



The visualizations above show the distribution of key property related numerical features. The upper row represents categorical numerical attributes (bedroom, bathroom, and carport counts), while the lower rows represent continuous attributes (price, land size, and building size) that have been log-transformed to normalize their scales.

1. Bedroom, Bathroom, and Carport: Most properties have 2–4 bedrooms, 1–3 bathrooms, and 0–2 carports, indicating that the dataset mainly consists of small to medium-sized houses. A few listings have higher values, suggesting large properties or potential outliers.
2. Price (log-transformed): After applying a logarithmic transformation, the price distribution appears approximately normal. This suggests that most properties fall within a moderate price range, while only a few have extremely high prices.
3. Land Size and Building Size (log-transformed): Both distributions are slightly right-skewed even after transformation, meaning there are a few properties with significantly larger land and building areas.

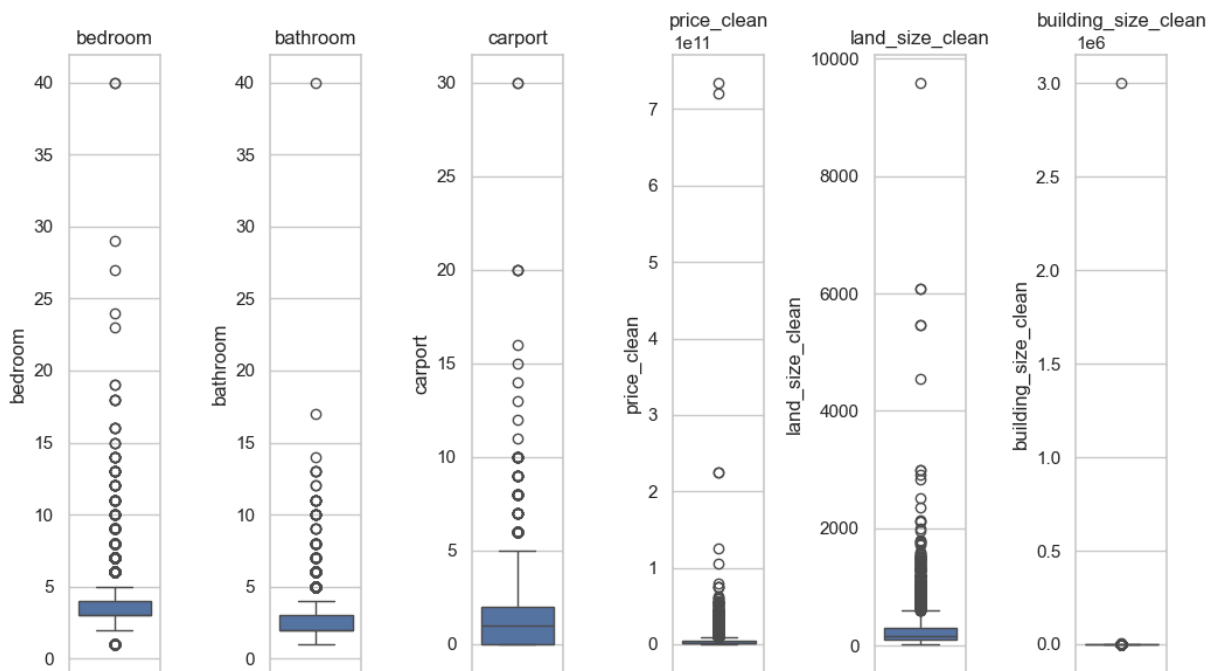
Overall, the dataset is dominated by moderately sized, residential houses, with only a small number of luxury or oversized listings.

Data Preprocessing Part 2

Outlier

```
In [26]: # Check for outlier
numeric_cols = df.columns

plt.figure(figsize=(12, 6))
for i, col in enumerate(numeric_cols, 1):
    plt.subplot(1, 7, i)
    sns.boxplot(y=df[col])
    plt.title(col)
plt.tight_layout()
plt.show()
```



```
In [27]: # Remove outlier using IQR
cols = df.columns

Q3 = df[cols].quantile(0.75)
Q1 = df[cols].quantile(0.25)

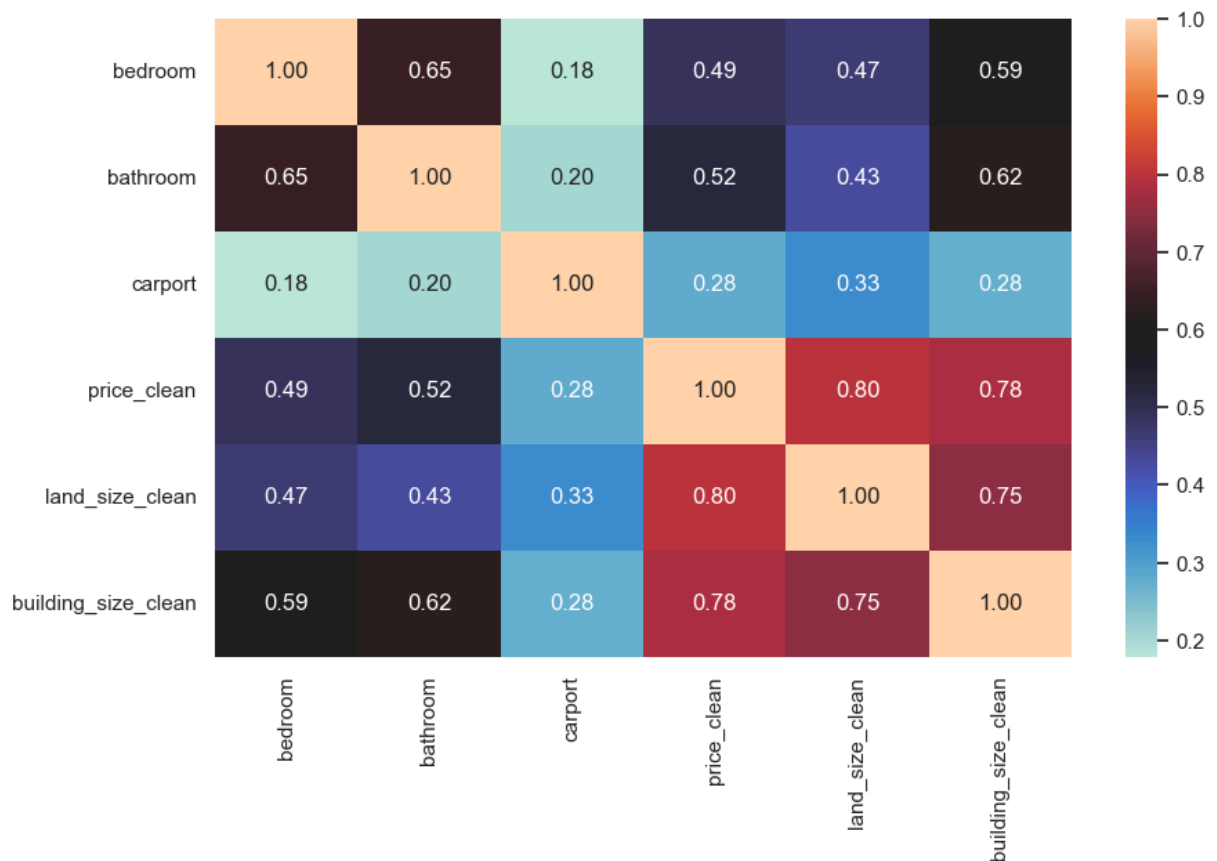
IQR = Q3 - Q1

df = df[~((df[cols] < (Q1 - 1.5 * IQR)) | (df[cols] > (Q3 + 1.5 * IQR))).any(axis=1)]
```

Correlation Matrix Heatmap

```
In [28]: plt.figure(figsize=(10,6))
sns.heatmap(df.corr(), annot=True, cmap='icefire', fmt=".2f")
```

Out[28]: <Axes: >



Train Test Split

```
In [29]: from sklearn.model_selection import train_test_split

#Train Test Split
X_train, X_test, y_train, y_test = train_test_split(df.drop(columns='price_clean'),
```

Model Building

Following models to predict the house price:

1. Linear Regression
2. Random Forest Regression
3. XGBoost Regressor

Linear Regression

Pipeline

```
In [30]: from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
```

```

from datetime import datetime

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LinearRegression())
])

# Start timer
start = datetime.now()
print(f"[INFO] started at {start.strftime('%H:%M:%S')}")

# Training the model
pipe.fit(X_train, y_train)

# End timer
end = datetime.now()
print(f"[INFO] ended at {end.strftime('%H:%M:%S')}")

# Compute duration
duration = end - start
total_seconds_lr = duration.total_seconds()
minutes = int(total_seconds_lr // 60)
seconds = int(total_seconds_lr % 60)

print(f"[INFO] Duration: {minutes} min {seconds} seconds")

# Predicting the values
pipe_pred = pipe.predict(X_test)

```

```

[INFO] started at 20:29:31
[INFO] ended at 20:29:31
[INFO] Duration: 0 min 0 seconds

```

In [31]: `from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score`

```

# Test Accuracy
mae_lr = mean_absolute_error(y_test, pipe_pred)
mse_lr = mean_squared_error(y_test, pipe_pred)
rmse_lr = np.sqrt(mse_lr)
r2_lr = r2_score(y_test, pipe_pred)

print("Regression Performance Metrics:")
print(f"MAE : {mae_lr:.2f}")
print(f"RMSE : {rmse_lr:.2f}")
print(f"R² : {r2_lr:.4f}")

plt.figure(figsize=(12,5))

plt.subplot(1, 2, 1)
sns.scatterplot(x=y_test, y=pipe_pred, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color='red', 1)
plt.title('Actual vs Predicted Prices')
plt.xlabel('Actual Price')
plt.ylabel('Predicted Price')

plt.subplot(1, 2, 2)

```



```

residuals = y_test - pipe_pred
sns.histplot(residuals, bins=30, kde=True, color='steelblue')
plt.title('Residual (Prediction Error) Distribution')
plt.xlabel('Error (Actual - Predicted)')
plt.ylabel('Count')

plt.tight_layout()
plt.show()

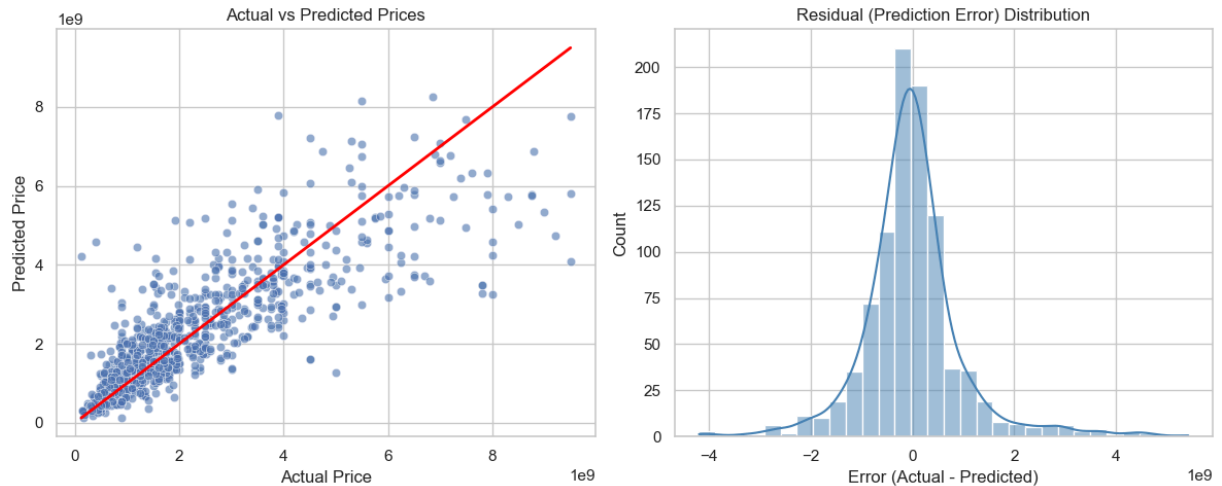
```

Regression Performance Metrics:

MAE : 639,423,963.56

RMSE : 983,560,649.94

R² : 0.6987



Random Forest Regression

Hyperparameter Tuning using RandomizedSearchCV

```

In [32]: from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
         from sklearn.ensemble import RandomForestRegressor

```

```

In [ ]: from datetime import datetime
         import numpy as np

         param_dist_rf = {
             'n_estimators': [200, 500, 800],
             'max_depth': [10, 20, 30],
             'max_features': ['sqrt', 0.8],
             'min_samples_split': [2, 5],
             'min_samples_leaf': [1, 2],
         }

         # Create a base model
         rf = RandomForestRegressor()

         # Create Randomized Search CV
         rf_random = RandomizedSearchCV(
             estimator=rf,
             param_distributions=param_dist_rf,

```

```

n_iter=10,
cv=3,
verbose=2,
random_state = 42,
n_jobs=-1
)

# Fit the model
rf_random.fit(X_train, y_train)

# Best parameters
print(rf_random.best_params_)

# Save best param + model to best_random_rf
best_random_rf = rf_random.best_estimator_

```

Fitting 3 folds for each of 10 candidates, totalling 30 fits

```

In [ ]: # Predicting the values
y_pred_rf = best_random_rf.predict(X_test)

# R2 Score (Model Accuracy)
r2_test = best_random_rf.score(X_test, y_test)

print(f"Testing R2: {r2_test:.4f}")

```

Hyperparameter Tuning using GridSearchCV

```

In [ ]: # Define the Grid Search parameter grid base on best param from Randomized Search
param_grid_rf = {
    'n_estimators': [700, 800, 900],
    'max_depth': [25, 30, 35],
    'max_features': [0.8],
    'min_samples_split': [2],
    'min_samples_leaf': [1,2],
}

# Create a base model
rf = RandomForestRegressor()

# Create Grid Search CV
rf_grid = GridSearchCV(
    estimator=rf,
    param_grid=param_grid_rf,
    cv=3,
    verbose=2,
    n_jobs=-1,
    scoring = 'r2'
)

# Train the model
rf_grid.fit(X_train, y_train)

# Best parameters
print(rf_grid.best_params_)

```

```
# Save best param + model to best_grid_rf
best_grid_rf = rf_grid.best_estimator_
```

```
In [ ]: # Start timer
start = datetime.now()
print(f"[INFO] started at {start.strftime('%H:%M:%S')}")

# Train the model
best_grid_rf.fit(X_train, y_train)

# End timer
end = datetime.now()
print(f"[INFO] ended at {end.strftime('%H:%M:%S')}")

# Compute duration
duration = end - start
total_seconds_rf = duration.total_seconds()
minutes = int(total_seconds_rf // 60)
seconds = int(total_seconds_rf % 60)
```

```
In [ ]: # Predicting the values
y_pred_rf = best_grid_rf.predict(X_test)

# R² Score (Model Accuracy)
r2_test = best_grid_rf.score(X_test, y_test)

print(f"Testing R²: {r2_test:.4f}")
```

```
In [ ]: # Test Accuracy
mae_rf = mean_absolute_error(y_test, y_pred_rf)
mse_rf = mean_squared_error(y_test, y_pred_rf)
rmse_rf = np.sqrt(mse_rf)
r2_rf = r2_score(y_test, y_pred_rf)

print("Random Forest Performance Metrics:")
print(f"MAE : {mae_rf:.2f}")
print(f"RMSE : {rmse_rf:.2f}")
print(f"R² : {r2_rf:.4f}")

plt.figure(figsize=(12,5))

plt.subplot(1, 2, 1)
sns.scatterplot(x=y_test, y=y_pred_rf, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color='red', 1)
plt.title('Actual vs Predicted Prices')
plt.xlabel('Actual Price')
plt.ylabel('Predicted Price')

plt.subplot(1, 2, 2)
residuals = y_test - y_pred_rf
sns.histplot(residuals, bins=30, kde=True, color='steelblue')
plt.title('Residual (Prediction Error) Distribution')
plt.xlabel('Error (Actual - Predicted)')
plt.ylabel('Count')
```

```
plt.tight_layout()
plt.show()
```

XGBoost Regression

Hyperparameter Tuning using RandomizedSearchCV

```
In [ ]: from xgboost import XGBRegressor
```

```
In [ ]: # Define the Randomized Search parameter grid
param_grid_xgb = {
    'n_estimators': [200, 500, 800],
    'max_depth': [5, 10, 15],
    'min_child_weight': [1, 3],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.8, 1.0],
    'learning_rate': [0.05, 0.1],
}

# Create a base model
xgb = XGBRegressor()

# Create RandomizedSearch CV
xgb_random = RandomizedSearchCV(
    estimator=xgb,
    param_distributions=param_grid_xgb,
    n_iter=10,
    cv=3,
    verbose=2,
    random_state=42,
    n_jobs=-1
)

# Train the model
xgb_random.fit(X_train, y_train)

# Best parameters
print(xgb_random.best_params_)

# Save best param + model
best_random_xgb = xgb_random.best_estimator_
```

```
In [ ]: # Prediksi data test
y_pred_rf = best_random_xgb.predict(X_test)

# R² Score (Model Accuracy)
r2_test = best_random_xgb.score(X_test, y_test)

print(f"Testing R²: {r2_test:.4f}")
```

Hyperparameter Tuning using GridSearch CV

```

In [ ]: # Define the Grid Search parameter grid base on best param from Randomized Search
param_grid_xgb = {
    'learning_rate': [0.03, 0.05, 0.1],
    'max_depth': [6, 8, 10],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.8],
    'min_child_weight': [1, 2],
    'n_estimators': [100, 200, 300]
}

# Create a base model
xgb = XGBRegressor()

# Create Grid Search CV
xgb_grid = GridSearchCV(
    estimator=xgb,
    param_grid=param_grid_xgb,
    cv=3,
    verbose=2,
    n_jobs=-1,
    scoring = 'r2'
)

# Fit the model
xgb_grid.fit(X_train, y_train)

# Best parameters
print(xgb_grid.best_params_)

# Save best param + model
best_grid_xgb = xgb_grid.best_estimator_

```

```

In [ ]: # Start timer
start = datetime.now()
print(f"[INFO] started at {start.strftime('%H:%M:%S')}")

# Train the model
best_grid_xgb.fit(X_train, y_train)

# End timer
end = datetime.now()
print(f"[INFO] ended at {end.strftime('%H:%M:%S')}")

# Compute duration
duration = end - start
total_seconds_xgb = duration.total_seconds()
minutes = int(total_seconds_xgb // 60)
seconds = int(total_seconds_xgb % 60)

```

```

In [ ]: # Prediksi data test
y_pred_xgb = best_grid_xgb.predict(X_test)

# R2 Score (Model Accuracy)
r2_test = best_grid_xgb.score(X_test, y_test)

```

```
print(f"Testing R²: {r2_test:.4f}")
```

```
In [ ]: # Test Accuracy
mae_xgb = mean_absolute_error(y_test, y_pred_xgb)
mse_xgb = mean_squared_error(y_test, y_pred_xgb)
rmse_xgb = np.sqrt(mse_xgb)
r2_xgb = r2_score(y_test, y_pred_xgb)

print("XGBoost Performance Metrics:")
print(f"MAE : {mae_xgb:.2f}")
print(f"RMSE : {rmse_xgb:.2f}")
print(f"R² : {r2_xgb:.4f}")

plt.figure(figsize=(12,5))

plt.subplot(1, 2, 1)
sns.scatterplot(x=y_test, y=y_pred_xgb, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color='red', 1
plt.title('Actual vs Predicted Prices')
plt.xlabel('Actual Price')
plt.ylabel('Predicted Price')

plt.subplot(1, 2, 2)
residuals = y_test - y_pred_xgb
sns.histplot(residuals, bins=30, kde=True, color='steelblue')
plt.title('Residual (Prediction Error) Distribution')
plt.xlabel('Error (Actual - Predicted)')
plt.ylabel('Count')

plt.tight_layout()
plt.show()
```

Model Evaluation

Regression Evaluation Metrics (MAE, RMSE, R2)

```
In [ ]: # Linear Regression
mae_lr = mean_absolute_error(y_test, pipe_pred)
mse_lr = mean_squared_error(y_test, pipe_pred)
rmse_lr = np.sqrt(mse_lr)
r2_lr = r2_score(y_test, pipe_pred)

# Random Forest Regression
mae_rf = mean_absolute_error(y_test, y_pred_rf)
mse_rf = mean_squared_error(y_test, y_pred_rf)
rmse_rf = np.sqrt(mse_rf)
r2_rf = r2_score(y_test, y_pred_rf)

# XGBoost Regression
mae_xgb = mean_absolute_error(y_test, y_pred_xgb)
mse_xgb = mean_squared_error(y_test, y_pred_xgb)
```

```
rmse_xgb = np.sqrt(mse_xgb)
r2_xgb = r2_score(y_test, y_pred_xgb)

results = pd.DataFrame({
    'Model': ['Linear Regression', 'Random Forest', 'XGBoost'],
    'MAE': [mae_lr, mae_rf, mae_xgb],
    'RMSE': [rmse_lr, rmse_rf, rmse_xgb],
    'R²': [r2_lr, r2_rf, r2_xgb]
})

results['MAE'] = results['MAE'].apply(lambda x: f"{x:.2f}")
results['RMSE'] = results['RMSE'].apply(lambda x: f"{x:.2f}")
results['R²'] = results['R²'].apply(lambda x: f"{x:.4f}")

display(results)
```

Probability Scatter Plot (Actual vs Predicted)

```
In [ ]: fig, ax = plt.subplots(1, 3, figsize=(20, 5))

# Linear Regression
sns.scatterplot(x=y_test, y=pipe_pred, alpha=0.6, ax=ax[0])
ax[0].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color='red',
ax[0].set_title('Linear Regression Scatter Plot')
ax[0].set_xlabel('Actual Price')
ax[0].set_ylabel('Predicted Price')

# Random Forest Regression
sns.scatterplot(x=y_test, y=y_pred_rf, alpha=0.6, ax=ax[1])
ax[1].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color='red',
ax[1].set_title('Random Forest Regression Scatter Plot')
ax[1].set_xlabel('Actual Price')
ax[1].set_ylabel('Predicted Price')

# XGBoost Regression
sns.scatterplot(x=y_test, y=y_pred_xgb, alpha=0.6, ax=ax[2])
ax[2].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color='red',
ax[2].set_title('XGBoost Regression Scatter Plot')
ax[2].set_xlabel('Actual Price')
ax[2].set_ylabel('Predicted Price')

plt.tight_layout()
plt.show()
```

The scatter plots above illustrate the relationship between actual and predicted house prices for three regression models: Linear Regression, Random Forest, and XGBoost.

All three models demonstrate a clear positive correlation between actual and predicted prices, with most data points clustering along the red diagonal line that represents perfect prediction. However, variations in the spread and alignment of points indicate differences in model precision and behavior.

Linear Regression effectively captures the overall price trend but tends to underpredict higher-priced houses, reflecting its limitation in modeling complex nonlinear relationships. Random Forest Regression better captures nonlinear patterns, reducing large errors though still showing slight overestimation in certain price ranges. XGBoost Regression provides the most balanced performance, maintaining tight alignment with the ideal line across nearly all price levels, indicating strong calibration and minimal bias.

Overall, the visualization suggests that XGBoost achieves the most accurate and stable predictions, followed by Random Forest, while Linear Regression remains a simpler yet reliable baseline.

Probability Distribution Plot (KDE Plot)

```
In [ ]: fig, ax = plt.subplots(1, 3, figsize=(20, 5))

# Logistic Regression
sns.kdeplot(y_test, color="r", label="Actual Value", ax=ax[0])
sns.kdeplot(pipe_pred, color="b", label="Fitted Values", ax=ax[0])
ax[0].set_title('Linear Regression')
ax[0].set_xlabel('price_clean')
ax[0].set_ylabel('Density')

# Random Forest
sns.kdeplot(y_test, color="r", label="Actual Value", ax=ax[1])
sns.kdeplot(y_pred_rf, color="b", label="Fitted Values", ax=ax[1])
ax[1].set_title('Random Forest')
ax[1].set_xlabel('price_clean')
ax[1].set_ylabel('Density')

# XGBoost
sns.kdeplot(y_test, color="r", label="Actual Value", ax=ax[2])
sns.kdeplot(y_pred_xgb, color="b", label="Fitted Values", ax=ax[2])
ax[2].set_title('XGBoost')
ax[2].set_xlabel('price_clean')
ax[2].set_ylabel('Density')

plt.tight_layout()
plt.show()
```

The following distribution plots compare the actual and predicted house price distributions for each regression model. Among the three, Random Forest Regression achieves the closest alignment with the actual distribution, indicating superior calibration and accuracy across most price ranges. XGBoost Regression also performs well but shows a slightly narrower peak, suggesting mild bias in specific regions. In contrast, Linear Regression produces a broader deviation, especially at higher price levels, reflecting its limitation in modeling nonlinear price dynamics compared to the ensemble-based models.

Error Evaluation (Residual Distribution / Error vs Predicted Plot)

```
In [ ]: # Residual Distribution Comparison
fig, ax = plt.subplots(1, 3, figsize=(20, 5))

# Linear Regression
residuals_lr = y_test - pipe_pred
sns.histplot(residuals_lr, bins=30, kde=True, color='steelblue', ax=ax[0])
ax[0].set_title('Linear Regression Residual Distribution')
ax[0].set_xlabel('Error (Actual - Predicted)')
ax[0].set_ylabel('Count')

# Random Forest Regression
residuals_rf = y_test - y_pred_rf
sns.histplot(residuals_rf, bins=30, kde=True, color='steelblue', ax=ax[1])
ax[1].set_title('Random Forest Residual Distribution')
ax[1].set_xlabel('Error (Actual - Predicted)')
ax[1].set_ylabel('Count')

# XGBoost Regression
residuals_xgb = y_test - y_pred_xgb
sns.histplot(residuals_xgb, bins=30, kde=True, color='steelblue', ax=ax[2])
ax[2].set_title('XGBoost Residual Distribution')
ax[2].set_xlabel('Error (Actual - Predicted)')
ax[2].set_ylabel('Count')

plt.tight_layout()
plt.show()
```

The residual plots above illustrate the distribution of prediction errors for the three regression models. All models show residuals centered around zero, indicating unbiased predictions overall. Random Forest and XGBoost exhibit narrower and more symmetrical distributions, suggesting greater stability and consistency in their predictions. In contrast, Linear Regression shows a slightly wider spread, implying higher variance and limited ability to capture nonlinear patterns. Overall, Random Forest and XGBoost demonstrate stronger residual performance and better generalization compared to Linear Regression.

Train Time Plot

```
In [ ]: plt.figure(figsize=(5,8))

models = ['Linear Regression', 'Random Forest', 'XGBoost']
times = [total_seconds_lr, total_seconds_rf, total_seconds_xgb]

bars = plt.bar(models, times, color=['#1f77b4', '#ff7f0e', '#2ca02c'])
plt.title("Model Training Time Comparison", fontsize=13)
plt.ylabel("Training Time (seconds)")

# Tambahkan angka di atas batang
```

```

for bar, time in zip(bars, times):
    plt.text(bar.get_x() + bar.get_width()/2, time + 0.05, f"{time:.2f}s",
             ha='center', va='bottom', fontsize=10)

# Relative training time (time ratio)
baseline = total_seconds_lr
ratios = [t / baseline for t in times]

for m, r in zip(models, ratios):
    if r > 1:
        print(f"{m}: {r:.2f}x slower than Linear Regression")
    elif r < 1:
        print(f"{m}: {1/r:.2f}x faster than Linear Regression")
    else:
        print(f"{m}: baseline (same as Linear Regression)")

plt.grid(axis='y', linestyle='--', alpha=0.4)
plt.tight_layout()
plt.show()

```

Feature Importance

```

In [ ]: # Setup style
sns.set(style="whitegrid", font_scale=1.1)

fig, ax = plt.subplots(1, 2, figsize=(20, 8))

# Random Forest Feature Importance
rf_features = pd.DataFrame({
    'Features': X_train.columns,
    'Importance': best_grid_rf.feature_importances_
})
rf_features.sort_values('Importance', ascending=False, inplace=True)

sns.barplot(
    x='Importance',
    y='Features',
    data=rf_features,
    palette="Blues_r",
    hue='Features',
    legend=False,
    ax=ax[0]
)
ax[0].set_title('Random Forest Feature Importance')
ax[0].set_xlabel('Importance')
ax[0].set_ylabel('Features')

# XGBoost Feature Importance
xgb_features = pd.DataFrame({
    'Features': X_train.columns,
    'Importance': best_grid_xgb.feature_importances_
})
xgb_features.sort_values('Importance', ascending=False, inplace=True)

```

```

sns.barplot(
    x='Importance',
    y='Features',
    data=xgb_features,
    palette="Oranges_r",
    hue='Features',
    legend=False,
    ax=ax[1]
)
ax[1].set_title('XGBoost Feature Importance')
ax[1].set_xlabel('Importance')
ax[1].set_ylabel('')

# Layout
plt.tight_layout()
plt.show()

```

The feature importance plots show that both Random Forest and XGBoost identify land size and building size as the most influential factors in predicting house prices. While Random Forest relies more heavily on land size, XGBoost distributes importance more evenly and gives slightly higher weight to features like bathroom, carport, and bedroom, reflecting better capture of nonlinear effects. Overall, property dimensions remain the strongest predictors across both models.

Predicting the price of new house using trained model

```

In [ ]: # Predicting prices for multiple sample houses
# Data Extracted from real-world listing (rumah123.com)
houses = pd.DataFrame([
    {
        'name': 'Rumah 1',
        'actual_price': 2500000000,
        'bedroom': 3,
        'bathroom': 2,
        'carport': 1,
        'land_size_clean': 188,
        'building_size_clean': 110
    },
    {
        'name': 'Rumah 2',
        'actual_price': 2250000000,
        'bedroom': 3,
        'bathroom': 3,
        'carport': 1,
        'land_size_clean': 183,
        'building_size_clean': 300
    },
    {
        'name': 'Rumah 3',
        'actual_price': 850000000,

```

```

        'bedroom': 2,
        'bathroom': 2,
        'carport': 0,
        'land_size_clean': 88,
        'building_size_clean': 70
    },
    {
        'name': 'Rumah 4',
        'actual_price': 966000000,
        'bedroom': 4,
        'bathroom': 3,
        'carport': 1,
        'land_size_clean': 60,
        'building_size_clean': 90
    },
    {
        'name': 'Rumah 5',
        'actual_price': 1880000000,
        'bedroom': 3,
        'bathroom': 2,
        'carport': 0,
        'land_size_clean': 126,
        'building_size_clean': 93
    }
])

# Feature for prediction
X_new = houses[['bedroom', 'bathroom', 'carport', 'land_size_clean', 'building_size

# Make Prediction
houses['Pred_Linear'] = pipe.predict(X_new)
houses['Pred_RF'] = best_grid_rf.predict(X_new)
houses['Pred_XGB'] = best_grid_xgb.predict(X_new)

# Format Price
def format_rupiah(x):
    return f"Rp {x:,.0f}".replace(",", ".")

houses['Actual_Price'] = houses['actual_price'].apply(format_rupiah)
houses['Linear_Price_Prediction'] = houses['Pred_Linear'].apply(format_rupiah)
houses['RandomForest_Price_Prediction'] = houses['Pred_RF'].apply(format_rupiah)
houses['XGBoost_Price_Prediction'] = houses['Pred_XGB'].apply(format_rupiah)

# Tampilkan hasil
houses_display = houses[['name', 'Actual_Price', 'Linear_Price_Prediction', 'Random
display(houses_display)

```

Conclusion

From the analysis, it can be observed that the XGBoost Regression model achieved the highest overall accuracy, with an R2 score of 0.7838, slightly outperforming Random Forest Regression (0.7765). However, further evaluation through residual and distribution plots revealed that Random Forest produced more stable and well-calibrated predictions, with

error values more tightly centered around zero and smoother alignment with the actual price distribution.

During model testing using unseen real-world data, the Random Forest model generated price predictions closer to the actual property listings, demonstrating stronger generalization and lower variance compared to XGBoost. This indicates that while XGBoost captures nonlinear relationships more aggressively, it may be slightly overfitted to the training data, whereas Random Forest offers a more balanced and consistent performance.

In conclusion, both ensemble models significantly outperform Linear Regression, with XGBoost excelling in in-sample accuracy and Random Forest standing out for its real-world reliability and stable predictive behavior. These findings suggest that Random Forest serves as a dependable baseline for automated house price estimation systems, while XGBoost can be leveraged for further optimization with careful regularization and feature refinement.